

A Comparison of Stochastic Gradient Descent, Scaled Conjugate Gradient and LeapFrog Training of Feedforward Neural Networks

Logan Shelver

Student number: 26946777

RW741 Machine Learning

Department of Computer Science, Stellenbosch University

26946777@sun.ac.za

Abstract—Training a feedforward neural network is an unconstrained minimisation of an error defined on the network weights. The study reported here compares stochastic gradient descent, scaled conjugate gradient, and the LeapFrog algorithm LFOP1(b) on three classification problems and three function-approximation problems of increasing complexity. A single hidden layer is used. The hidden-unit count and the control parameters of each trainer are selected by validation error. A total of 30 independent runs and nonparametric statistical tests are used to rank the trainers. Scaled conjugate gradient obtained the lowest mean test error on four of the six problems. The three trainers were statistically indistinguishable on the Glass Identification problem.

Index Terms—neural networks, stochastic gradient descent, scaled conjugate gradient, LeapFrog, classification, function approximation

I. INTRODUCTION

A feedforward neural network is trained by minimising a scalar error $E(\mathbf{w})$ over a packed weight vector $\mathbf{w}$. The minimiser determines both the attained training error and the generalisation of the trained network. First-order methods such as stochastic gradient descent (SGD) are simple to implement [8], [12]. The success of SGD depends on the learning rate, the momentum coefficient, and the mini-batch size [8], [12]. Second-order methods and dynamic methods reduce the dependence on the learning rate, the momentum coefficient, and the mini-batch size [10], [14].

Møller introduced scaled conjugate gradient (SCG) as a conjugate-gradient trainer that avoids a line search by Levenberg–Marquardt scaling of a Hessian-vector product [10]. Snyman introduced LeapFrog as a dynamic unconstrained minimiser that treats E as a potential energy [14]. The leap-frog optimiser (LFOP) variant LFOP1(b) adds an adaptive time step [15]. Engelbrecht discussed LeapFrog as a neural-network training algorithm [2].

The study reported here compares SGD, SCG, and LeapFrog LFOP1(b) as trainers of a one-hidden-layer feedforward network. Three classification problems and three function-approximation problems of increasing complexity are used. An empirical search determines the hidden-unit count of each

problem. A second empirical search determines the control parameters of each trainer. A total of 30 independent initialisations and nonparametric tests are used to rank the trainers.

The remainder of the report is organised as follows. Section II describes the network model and the three training algorithms. Section III records the implementation choices. Section IV defines the empirical process. Section V reports and discusses the results. Section VI concludes the report.

II. BACKGROUND

The three trainers compared in the study rest on different principles. Section II-A describes the network. Section II-B describes stochastic gradient descent. Section II-C describes scaled conjugate gradient. Section II-D describes LeapFrog. Section II-E then defines the performance measures and the statistical tests.

A. Feedforward neural networks

A one-hidden-layer network maps an input $\mathbf{x} \in \mathbb{R}^{n_i}$ to an output $\mathbf{y} \in \mathbb{R}^{n_o}$. The hidden pre-activation and the hidden activation are

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{h} = \tanh(\mathbf{z}^{(1)}), \quad (1)$$

where $\mathbf{W}^{(1)} \in \mathbb{R}^{n_i \times n_h}$ and $\mathbf{b}^{(1)} \in \mathbb{R}^{n_h}$. The output pre-activation is

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)\top} \mathbf{h} + \mathbf{b}^{(2)}. \quad (2)$$

Classification uses a softmax output. Function approximation uses a linear output. All weights and biases are packed into a single vector $\mathbf{w} \in \mathbb{R}^N$. Training is then unconstrained minimisation of $E(\mathbf{w})$. Bishop treated the packed-vector view of supervised learning as standard [1].

The error E is assumed continuously differentiable. Reverse-mode backpropagation therefore yields $\nabla E(\mathbf{w})$. Rumelhart *et al.* derived the backpropagation recurrence [12].

B. Stochastic gradient descent

SGD replaces $\nabla E(\mathbf{w})$ by a gradient estimated on a mini-batch B of the training set. With momentum coefficient $\beta \in [0, 1)$ and learning rate $\eta > 0$, the update is

$$\mathbf{u} \leftarrow \beta \mathbf{u} - \eta \nabla E_B(\mathbf{w}), \quad \mathbf{w} \leftarrow \mathbf{w} + \mathbf{u}. \quad (3)$$

Polyak introduced the momentum term as a heavy-ball correction [11]. A batch size equal to the training-set size recovers full-batch gradient descent. The learning rate, the momentum coefficient, and the batch size are control parameters [8]. The control parameters are problem dependent.

C. Scaled conjugate gradient

Conjugate-gradient methods build search directions that are conjugate with respect to the Hessian of a quadratic model of E . A line search is then required to choose a step length. Møller avoided the line search by combining a conjugate-gradient direction with a Levenberg–Marquardt scalar λ_k that keeps the model positive definite [10].

Let $\mathbf{r}_k = -\nabla E(\mathbf{w}_k)$ and let $\mathbf{p}_k$ be the search direction. A finite-difference Hessian-vector product

$$\mathbf{q}_k = \frac{\nabla E(\mathbf{w}_k + \sigma_k \mathbf{p}_k) - \nabla E(\mathbf{w}_k)}{\sigma_k}, \quad \sigma_k = \frac{\sigma}{\|\mathbf{p}_k\|}, \quad (4)$$

is scaled to $\delta_k = \mathbf{p}_k^\top \mathbf{q}_k + (\lambda_k - \bar{\lambda}_k) \|\mathbf{p}_k\|^2$. If $\delta_k \leq 0$, the scale λ_k is raised until the model is positive definite. The step length is $\alpha_k = \mu_k / \delta_k$ with $\mu_k = \mathbf{p}_k^\top \mathbf{r}_k$. A comparison parameter Δ_k measures how well the quadratic model predicts the true reduction in E . A successful step is accepted. The scale λ_k is lowered when $\Delta_k \geq 0.75$. The scale λ_k is raised when $\Delta_k < 0.25$. The search direction is then formed with the Polak–Ribière formula, or the search is restarted along $\mathbf{r}_{k+1}$. Møller presented $\sigma \leq 10^{-4}$ and $\lambda_1 \leq 10^{-6}$ as values that are not critical for success [10]. The present study still tunes σ and λ_1 on a validation grid.

D. LeapFrog unconstrained minimisation

Snyman modelled unconstrained minimisation as the motion of a particle of unit mass in the potential E [14]. The acceleration is $\mathbf{a} = -\nabla E(\mathbf{w})$. A leap-frog integrator advances the position and the velocity,

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \mathbf{v}_k \Delta t, \quad \mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_{k+1} \Delta t. \quad (5)$$

The displacement of a step is capped by a maximum length $\delta_{\max}$. If the kinetic energy decreases, the particle is placed at the midpoint of the last step. The velocity is then reduced, or set to zero after repeated decreases. The LFOP1(b) modification grows Δt after consecutive successful steps and halves Δt after m consecutive obtuse angles between successive accelerations [15]. Snyman used $\Delta t = 0.5$, $m = 3$, and compared $\delta_{\max} = 1$ with $\delta_{\max} = 100$ [15]. The present study tunes Δt and $\delta_{\max}$ on a validation grid and keeps $m = 3$.

E. Performance measures

Classification is scored on a locked test set by accuracy, macro-averaged F_1 , and mean categorical cross-entropy. Function approximation is scored by mean squared error (MSE), root mean squared error, and the coefficient of determination R^2 . Training effort is recorded as the number of gradient evaluations and as wall-clock time. The ranking of trainers uses test accuracy on classification problems and test MSE on function-approximation problems.

F. Statistical comparison of stochastic trainers

Random initial weights make each run a random variable. Shapiro and Wilk tested normality of a sample [13]. If every sample of 30 test scores is consistent with normality, a one-way analysis of variance is used. Kruskal and Wallis provided a nonparametric alternative when normality did not hold [7]. Pairwise differences are tested with the Mann–Whitney procedure [9]. Holm correction controls the family-wise error of the three pairwise tests on each problem [6]. Friedman ranking across problems is used as a further summary [4]. A significance level of 0.05 is used throughout.

SGD, SCG, and LeapFrog LFOP1(b) minimise the same differentiable error $E(\mathbf{w})$. SGD is a first-order mini-batch method with three control parameters. SCG is a scaled conjugate-gradient method with a finite-difference Hessian-vector product. LeapFrog is a damped dynamic trajectory method with an adaptive time step.

III. IMPLEMENTATION

The implementation records the architecture, the cost functions, the preprocessing, the three trainers, and the stopping rules. Section III-A describes the network. Section III-B describes the cost functions. Section III-C describes the preprocessing. Section III-D through Section III-G describe the trainers and the stopping rules.

A. Network architecture and initialisation

One hidden layer of hyperbolic-tangent units was used on every problem. Classification used a softmax output. Function approximation used a linear output. Glorot and Bengio proposed the uniform Xavier distribution matched to hyperbolic-tangent units [5]. Weights were drawn from the uniform Xavier distribution. Biases were initialised at zero. A packed vector $\mathbf{w}$ concatenates $\mathbf{W}^{(1)}$, $\mathbf{b}^{(1)}$, $\mathbf{W}^{(2)}$, and $\mathbf{b}^{(2)}$ in that order.

B. Cost functions

Classification used mean categorical cross-entropy

$$E(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^{n_o} y_{ic} \log(\hat{y}_{ic} + \varepsilon), \quad (6)$$

with $\varepsilon = 10^{-12}$. Function approximation used MSE

$$E(\mathbf{w}) = \frac{1}{n n_o} \sum_{i=1}^n \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|^2. \quad (7)$$

The analytic gradient matched E . The gradient was checked against a central finite difference on exclusive-or. The relative error of the finite-difference check was 5.4×10^{-11} .

Algorithm 1 Mini-batch stochastic gradient descent

Require: packed weights $\mathbf{w}$, learning rate η , momentum β , batch size s

- 1: set the momentum buffer $\mathbf{u}$ to zero
- 2: **for** every epoch **do**
- 3: shuffle the training set
- 4: **for** every mini-batch B of size s **do**
- 5: set $\mathbf{u}$ to $\beta\mathbf{u} - \eta\nabla E_B(\mathbf{w})$
- 6: set $\mathbf{w}$ to $\mathbf{w} + \mathbf{u}$
- 7: **end for**
- 8: record the training error and the validation error
- 9: **end for**

C. Data preprocessing

Input features were standardised from the training split only. The training mean and the training standard deviation were applied to the validation split and to the test split. Classification targets were one-hot encoded. A linear output with MSE is sensitive to target scale. Regression targets were therefore standardised from the training split. Splits were stratified on classification problems. A random split was used on function-approximation problems. The outer proportions were 70% train, 15% validation, and 15% test, with a fixed seed. Every trainer therefore saw the same partitions.

D. Stochastic gradient descent implementation

Algorithm 1 states the mini-batch update. A full pass over the training set is one epoch. The validation error was recorded at the end of each epoch.

E. Scaled conjugate gradient implementation

Algorithm 2 follows the numbered steps of Møller [10]. Two implementation safeguards were added. A search direction that is not a descent direction was replaced by the steepest-descent vector. A failed reduction restarted conjugacy from steepest descent after λ_k was raised. The restart prevented a near-singular conjugate direction from stalling the iteration. The two safeguards did not change the scale-update formulae of Møller.

F. LeapFrog implementation

Algorithm 3 follows algorithm (23) of Snyman [14] with the LFOP1(b) time-step rules [15]. The time step was clipped to $[10^{-8}, 10]$ to prevent overflow after many consecutive successful steps.

G. Stopping criteria

Every trainer stopped at the first of three events: a validation-error plateau of 60 recorded steps, a gradient norm below 10^{-8} , or a cap of 20 000 gradient evaluations. The weights with the lowest validation error were restored. The same three events were used for all algorithms. A comparison of test error was therefore not confounded by unequal stopping rules.

A shared network and a shared analytic gradient were used by SGD, SCG, and LeapFrog. The three algorithms differed

Algorithm 2 Scaled conjugate gradient

Require: packed weights $\mathbf{w}$, interval σ , scale λ

- 1: set $\mathbf{r}$ to $-\nabla E(\mathbf{w})$, set $\mathbf{p}$ to $\mathbf{r}$, set $\bar{\lambda}$ to 0, and mark the iteration as successful
- 2: **while** the stopping rule is not met **do**
- 3: **if** the previous iteration succeeded **then**
- 4: estimate $\mathbf{q}$ and set δ_k to $\mathbf{p}^\top \mathbf{q}$
- 5: **end if**
- 6: set δ_k to $\delta_k + (\lambda - \bar{\lambda})\|\mathbf{p}\|^2$
- 7: **if** $\delta_k \leq 0$ **then**
- 8: raise λ until the quadratic model is positive definite
- 9: **end if**
- 10: set α to $(\mathbf{p}^\top \mathbf{r})/\delta_k$
- 11: compute Δ_k from the true reduction and the quadratic reduction
- 12: **if** $\Delta_k \geq 0$ **then**
- 13: set $\mathbf{w}$ to $\mathbf{w} + \alpha\mathbf{p}$, update $\mathbf{r}$, and accept the step
- 14: restart the direction or apply the Polak–Ribière formula for $\mathbf{p}$
- 15: **else**
- 16: reject the step, raise λ , and set $\mathbf{p}$ to $\mathbf{r}$
- 17: **end if**
- 18: **end while**

Algorithm 3 LeapFrog LFOP1(b)

Require: packed weights $\mathbf{w}$, time step Δt , maximum step $\delta_{\max}$, and $m = 3$

- 1: set $\mathbf{a}$ to $-\nabla E(\mathbf{w})$ and set $\mathbf{v}$ to $\frac{1}{2}\mathbf{a}\Delta t$
- 2: **while** the stopping rule is not met **do**
- 3: cap $\|\mathbf{v}\|\Delta t$ by $\delta_{\max}$
- 4: **if** the last m successive accelerations form obtuse angles **then**
- 5: halve Δt and restart from the midpoint with reduced velocity
- 6: **end if**
- 7: set $\mathbf{w}$ to $\mathbf{w} + \mathbf{v}\Delta t$, set $\mathbf{a}$ to $-\nabla E(\mathbf{w})$, and set $\mathbf{v}$ to $\mathbf{v} + \mathbf{a}\Delta t$
- 8: **if** kinetic energy decreased **then**
- 9: move to the midpoint and reduce or zero the velocity
- 10: **end if**
- 11: **end while**

only in the update of $\mathbf{w}$. Python 3 and NumPy were used. No third-party implementation of SCG or LeapFrog was used for the reported numbers.

IV. EMPIRICAL PROCESS

The empirical process covers the problems, the hidden-unit search, the control-parameter search, and the 30-run comparison. Section IV-A describes the problems. Section IV-B describes the hidden-unit search. Section IV-C describes the control-parameter search. Section IV-D describes the final comparison.

A. Problems

Three classification problems and three function-approximation problems were selected to form a complexity ladder. Table I records the dimensions after the split.

TABLE I
PROBLEM DIMENSIONS AFTER THE 70/15/15 SPLIT.

Problem	n_i	n_o	Train	Val.	Test
Iris	4	3	104	23	23
Breast Cancer	30	2	397	86	86
Glass	9	6	149	32	33
sinc	1	1	139	31	30
sincos	2	1	279	61	60
Friedman #1	10	1	279	61	60

The Iris data set has 150 samples, four continuous features, and three flower classes. Iris was treated as the easy classification problem. The Wisconsin Breast Cancer Diagnostic data set has 569 samples, 30 real-valued features, and two classes. Breast Cancer was treated as the medium classification problem. The Glass Identification data set has 214 samples, nine chemical features, and six imbalanced classes. Glass was treated as the hard classification problem. The three classification sets were taken from the University of California, Irvine (UCI) repository.¹

A one-dimensional sinc function f was treated as the easy function-approximation problem. The function f satisfies $f(x) = \sin(x)/x$ on $[-10, 10]$. A total of 200 noise-free samples were drawn. A two-dimensional function g was treated as the medium function-approximation problem. The function g satisfies $g(x, y) = \sin(x)\cos(y)$ on $[-\pi, \pi]^2$. A total of 400 noise-free samples were drawn. Friedman #1 was treated as the hard function-approximation problem [3]. The problem has ten inputs. Five inputs are informative. Five inputs are irrelevant. A total of 400 samples were drawn with Gaussian noise of standard deviation 0.1.

B. Hidden-unit search

Optimality of the hidden-unit count was defined as follows. Ten independent runs were performed at each candidate count. The selected count was the smallest count with a mean validation error within five percent of the best mean on the problem. The five-percent band preferred a smaller network when the validation error was statistically close to the best value. SCG has few control parameters. SCG converges in few iterations. SCG with $\sigma = 10^{-4}$ and $\lambda_1 = 10^{-6}$ was therefore used for the architecture search. The grid $\{2, 4, 6, 8, 12, 16\}$ was used on Iris and sinc. The grid $\{4, 8, 12, 16, 24\}$ was used on sincos. The grid $\{4, 8, 12, 16, 24, 32\}$ was used on Breast Cancer, Glass, and Friedman #1. The selected count was then frozen for every trainer on that problem.

C. Control-parameter search

After the architecture was locked, each trainer was tuned on each problem by mean validation error over eight runs. The SGD grid was $\eta \in \{0.001, 0.01, 0.1\}$, $\beta \in \{0.0, 0.9\}$, and batch size in $\{16, 32, n\}$. The SCG grid was $\sigma \in \{10^{-4}, 10^{-5}\}$ and $\lambda_1 \in \{10^{-6}, 10^{-4}\}$. The LeapFrog grid was $\Delta t \in \{0.1, 0.5, 1.0\}$ and $\delta_{\max} \in \{1, 10, 100\}$. The value of m was held at three. The winning tuple of each trainer on each problem was locked for the final comparison.

D. Final comparison

A total of 30 independent Xavier initialisations were drawn for each pair of algorithm and problem. The splits, the hidden-unit counts, and the control parameters were held fixed. Test accuracy was the primary classification score. Test MSE was the primary function-approximation score. Shapiro–Wilk tests were applied to the 30 primary scores. Kruskal–Wallis tests and Holm-corrected Mann–Whitney tests were then applied on each problem. A Friedman test ranked the three trainers across the six problems.

The protocol had four locked stages: problem construction, hidden-unit search, control-parameter search, and a 30-run comparison. Reported results were produced only from the locked protocol.

V. RESULTS AND DISCUSSION

The results address the hidden-unit search, the selection of the control parameters, and the comparison of the three trainers. Section V-A reports the hidden-unit search. Section V-B reports the tuning. Section V-C and Section V-D compare the trainers on classification and on function approximation. Section V-E reports the statistical tests.

A. Hidden-unit search

Table II records the selected hidden-unit counts.

TABLE II
SELECTED HIDDEN-UNIT COUNTS FROM THE VALIDATION SEARCH.

Problem	Hidden units	Val. error	Train error
Iris	4	0.0316	0.0340
Breast Cancer	32	0.0024	0.0000
Glass	4	0.7921	0.4507
sinc	8	0.0043	0.0036
sincos	24	0.0184	0.0114
Friedman #1	12	0.0262	0.0055

Iris selected four hidden units. Figure 1 shows the validation curve. The mean validation cross-entropy at four units was 0.0316. Larger nets did not improve on that value by more than five percent.

¹<https://archive.ics.uci.edu>

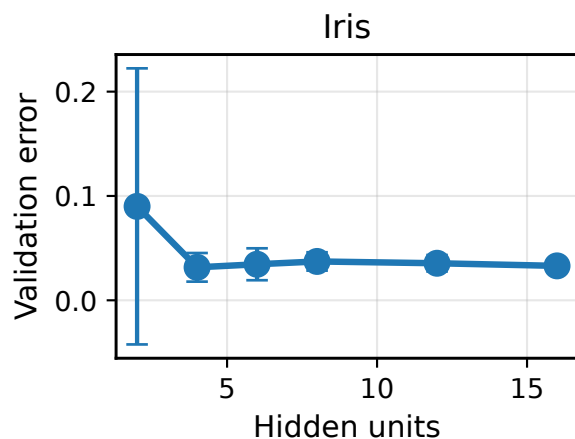


Fig. 1. Hidden-unit search on Iris.

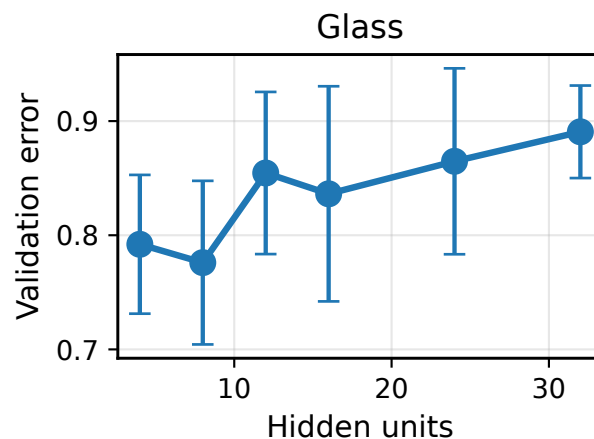


Fig. 3. Hidden-unit search on Glass.

Breast Cancer selected 32 hidden units. Figure 2 shows the validation curve. Training cross-entropy reached zero from eight units upward. Validation cross-entropy continued to fall. The five-percent rule therefore retained the largest grid value.

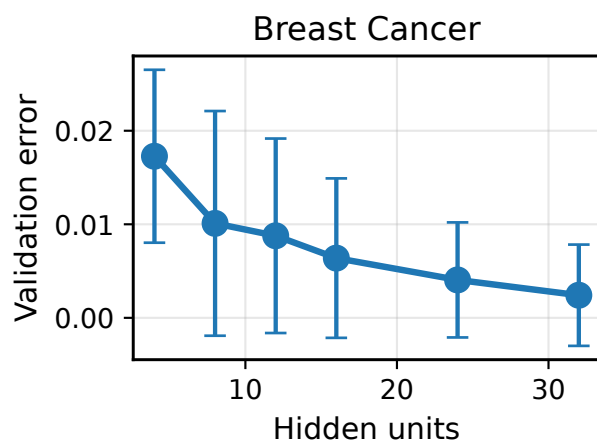


Fig. 2. Hidden-unit search on Breast Cancer.

The sinc problem selected eight hidden units. The mean validation MSE was 0.0043. Figure 4 shows the validation curve.

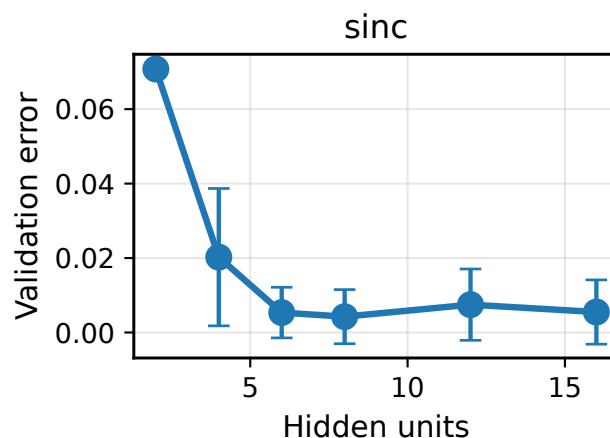


Fig. 4. Hidden-unit search on sinc.

Glass selected four hidden units. Figure 3 shows the validation curve. The lowest mean validation error on Glass was 0.776 at eight units. A count of four units lay inside the five-percent band and was therefore preferred. Training error on Glass kept falling while the hidden layer grew. Validation error rose. The pattern is the signature of overfitting on a small, imbalanced set.

The sincos problem selected 24 hidden units. Figure 5 shows the validation curve.

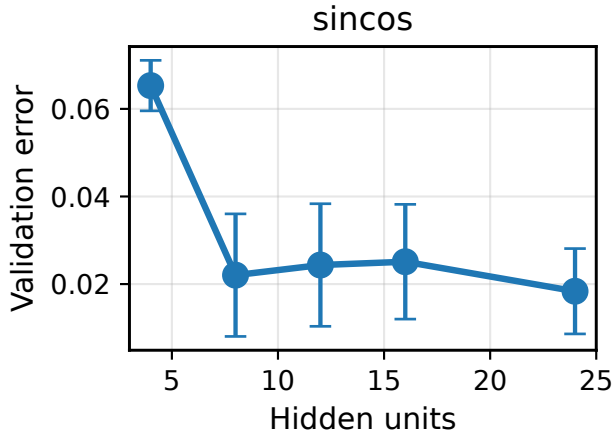


Fig. 5. Hidden-unit search on sincos.

Friedman #1 selected 12 hidden units. Figure 6 shows the validation curve. The mean validation MSE at 12 units was 0.0262. Both smaller nets and larger nets were worse.

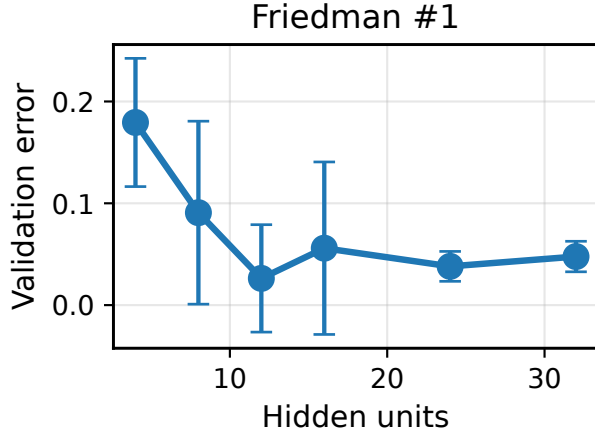


Fig. 6. Hidden-unit search on Friedman #1.

B. Control-parameter search

Table III records the winning control parameters and the mean validation error of each winner.

TABLE III
LOCKED CONTROL PARAMETERS AND MEAN VALIDATION ERROR.

Problem and trainer	Locked parameters	Val. error
Iris, SGD	$\eta = 0.1, \beta = 0.9$, batch 32	0.031
Iris, SCG	$\sigma = 10^{-5}, \lambda_1 = 10^{-4}$	0.023
Iris, LeapFrog	$\Delta t = 1.0, \delta_{\max} = 1$	0.070
Breast Cancer, SGD	$\eta = 0.1, \beta = 0.9$, batch 32	0.008
Breast Cancer, SCG	$\sigma = 10^{-4}, \lambda_1 = 10^{-6}$	0.001
Breast Cancer, LeapFrog	$\Delta t = 1.0, \delta_{\max} = 10$	0.006
Glass, SGD	$\eta = 0.1, \beta = 0.9$, batch 16	0.692
Glass, SCG	$\sigma = 10^{-4}, \lambda_1 = 10^{-6}$	0.846
Glass, LeapFrog	$\Delta t = 1.0, \delta_{\max} = 10$	0.830
sinc, SGD	$\eta = 0.1, \beta = 0.9$, batch 32	0.022
sinc, SCG	$\sigma = 10^{-4}, \lambda_1 = 10^{-4}$	0.004
sinc, LeapFrog	$\Delta t = 1.0, \delta_{\max} = 1$	0.010
sincos, SGD	$\eta = 0.01, \beta = 0.9$, batch 16	0.041
sincos, SCG	$\sigma = 10^{-5}, \lambda_1 = 10^{-4}$	0.017
sincos, LeapFrog	$\Delta t = 1.0, \delta_{\max} = 1$	0.035
Friedman #1, SGD	$\eta = 0.1, \beta = 0.0$, batch 16	0.074
Friedman #1, SCG	$\sigma = 10^{-5}, \lambda_1 = 10^{-6}$	0.017
Friedman #1, LeapFrog	$\Delta t = 1.0, \delta_{\max} = 1$	0.014

SGD preferred $\eta = 0.1$ and $\beta = 0.9$ on four of the six problems. Full-batch SGD was never selected. SCG preferred values at the defaults of Möller or at the neighbouring grid point on every problem. The observation agrees with the claim that σ and λ_1 are not critical [10]. LeapFrog preferred $\Delta t = 1.0$ on every problem. The maximum step $\delta_{\max}$ was 1 on Iris, sinc, sincos, and Friedman #1. The maximum step $\delta_{\max}$ was 10 on Breast Cancer and Glass. A value $\delta_{\max} = 100$ with $\Delta t = 1.0$ produced a mean validation error of 0.67 on sincos. An oversized step is therefore harmful.

On Glass, SGD obtained a mean validation cross-entropy of 0.692. SCG obtained 0.846 on Glass. LeapFrog obtained 0.830 on Glass. The architecture of four hidden units, chosen by SCG, is a small model of a six-class problem. The lower validation error of SGD on Glass is consistent with mini-batch regularisation on a small architecture.

C. Classification test performance

Figure 7 and Table IV report mean test accuracy over 30 runs.

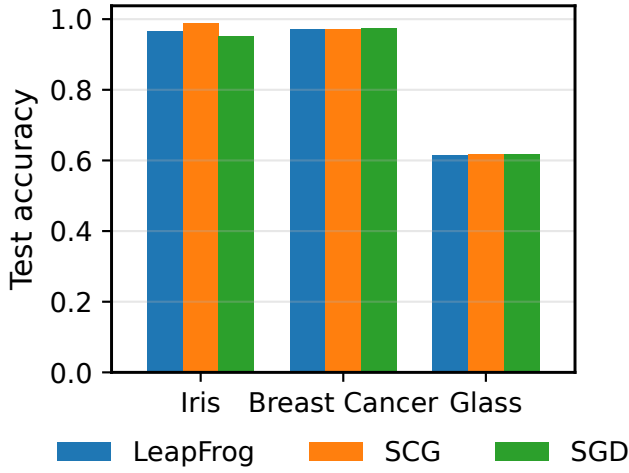


Fig. 7. Mean test accuracy on the classification problems.

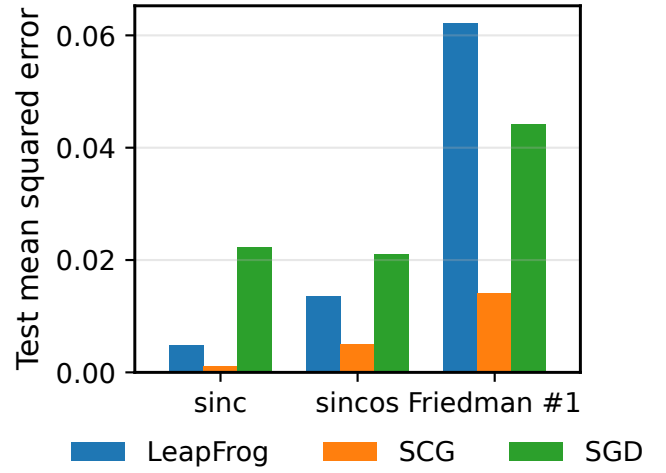


Fig. 8. Mean test mean squared error on the function-approximation problems.

TABLE IV
MEAN TEST ACCURACY $\pm$ STANDARD DEVIATION OVER 30 RUNS.

Problem	SGD	SCG	LeapFrog
Iris	0.951 ± 0.027	0.988 ± 0.022	0.965 ± 0.041
Breast Cancer	0.975 ± 0.004	0.973 ± 0.009	0.971 ± 0.007
Glass	0.617 ± 0.071	0.619 ± 0.072	0.615 ± 0.083

SCG obtained the highest mean accuracy on Iris, equal to 0.988. SGD obtained the highest mean accuracy on Breast Cancer, equal to 0.975. The three means on Breast Cancer lay inside one standard deviation of each other. Glass remained difficult for every trainer. Mean test accuracy was 0.617 for SGD, 0.619 for SCG, and 0.615 for LeapFrog. The standard deviations were 0.071, 0.072, and 0.083. Macro-averaged F_1 on Glass separated the trainers more clearly than accuracy. The macro-averaged F_1 was 0.499 for SCG, 0.484 for SGD, and 0.435 for LeapFrog. The small hidden layer, the six-class imbalance, and the 33-sample test set explained the high variance on Glass.

D. Function-approximation test performance

Figure 8 and Table V report mean test MSE over 30 runs.

TABLE V
MEAN TEST MEAN SQUARED ERROR $\pm$ STANDARD DEVIATION OVER 30 RUNS.

Problem	SGD	SCG	LeapFrog
sinc	0.022 ± 0.045	0.001 ± 0.004	0.005 ± 0.008
sincos	0.021 ± 0.007	0.005 ± 0.006	0.014 ± 0.008
Friedman #1	0.044 ± 0.045	0.014 ± 0.052	0.062 ± 0.099

SCG obtained the lowest mean test MSE on sinc at 0.0012, on sincos at 0.0051, and on Friedman #1 at 0.0141. LeapFrog was second on sinc and sincos. SGD had a large standard deviation on sinc of 0.045 relative to SCG at 0.004. The gap is consistent with residual sensitivity of SGD to random initialisation. LeapFrog was the weakest of the three methods on Friedman #1, with mean MSE 0.062 and standard deviation 0.099. Mean coefficients of determination on sinc were 0.998 for SCG, 0.993 for LeapFrog, and 0.966 for SGD. Irrelevant inputs and additive noise produced a less smooth error surface. The kinetic-energy heuristic of LeapFrog was less reliable on the Friedman #1 error surface than the scaled curvature of SCG.

E. Statistical comparison

Table VI records the omnibus test and the Holm-corrected pairwise decisions.

TABLE VI
OMNIBUS TESTS ON THE 30 PRIMARY TEST SCORES.

Problem	Test	p	Decision at 0.05
Iris	Kruskal–Wallis	3.5×10^{-5}	reject
Breast Cancer	Kruskal–Wallis	0.073	do not reject
Glass	analysis of variance	0.98	do not reject
sinc	Kruskal–Wallis	1.5×10^{-6}	reject
sincos	Kruskal–Wallis	4.5×10^{-11}	reject
Friedman #1	Kruskal–Wallis	2.5×10^{-10}	reject
All six means	Friedman	0.042	reject

Shapiro–Wilk tests rejected normality on five of the six problems. Kruskal–Wallis tests were therefore used on those problems. Glass was consistent with normality. A one-way analysis of variance was therefore used on Glass.

The Kruskal–Wallis test rejected equality of the three trainers on Iris, with $p = 3.5 \times 10^{-5}$. The same test rejected equality on sinc, with $p = 1.5 \times 10^{-6}$. The same test rejected equality on sincos, with $p = 4.5 \times 10^{-11}$. The same test rejected equality on Friedman #1, with $p = 2.5 \times 10^{-10}$. Holm-corrected Mann–Whitney tests found SCG better than SGD and better than LeapFrog on Iris, sinc, sincos, and Friedman #1. SGD was better than LeapFrog on sinc, sincos, and Friedman #1. Breast Cancer did not reject equality, with a Kruskal–Wallis p -value of 0.073. Glass did not reject equality, with an analysis-of-variance p -value of 0.98.

A Friedman test on the six mean primary scores rejected equality of ranks, with $p = 0.042$. The average ranks were 1.17 for SCG, 2.33 for SGD, and 2.50 for LeapFrog, with rank one denoting the best mean.

The ranking was therefore not uniform across problem types. On smooth, low-dimensional function approximation, SCG was the strongest of the three methods. On linearly well-separated medical data with 30 features, mini-batch SGD matched SCG. On a small imbalanced multi-class problem with a parsimonious hidden layer, no trainer dominated.

F. Discussion

Three observations follow from the tables. First, the claim of Møller that SCG has no critical user parameters is supported by the validation grid [10]. Neighbouring (σ, λ_1) pairs produced similar validation errors. Second, SGD remained competitive when η and β were tuned per problem. SGD used far more gradient evaluations than SCG and LeapFrog on every problem. Each mini-batch step is one evaluation. Early stopping is checked only once per epoch. Third, LeapFrog LFOP1(b) was an effective neural-network trainer on the easy and medium problems. Engelbrecht discussed LeapFrog for neural-network training [2]. LeapFrog was more sensitive to $\delta_{\max}$ than SCG was to σ . LeapFrog was the least stable of the three methods on Friedman #1.

A limitation of the study was the use of a single hidden layer and of small data sets. A second limitation was the architecture search with SCG only. A hidden-unit count that is optimal for SCG need not be optimal for SGD or LeapFrog. The protocol

matched the stated requirement of one hidden-layer size per problem, not per algorithm. SCG was the most reliable of the three trainers across the six problems. SGD matched SCG on Breast Cancer after a per-problem grid search. LeapFrog was competitive on classification and on the two smoother regression problems. LeapFrog was the weakest method on Friedman #1. Glass defeated all three trainers to a similar degree.

VI. CONCLUSIONS

Stochastic gradient descent (SGD), scaled conjugate gradient (SCG), and the LeapFrog algorithm LFOP1(b) were compared as trainers of a one-hidden-layer feedforward neural network. Six problems of increasing complexity were used. Hidden-unit counts and control parameters were selected by validation error. A total of 30 independent runs supplied the test scores.

SCG obtained the lowest mean test error on Iris, sinc, sincos, and Friedman #1. SGD obtained the lowest mean test accuracy on Breast Cancer. All three means on Breast Cancer were close. The three trainers were indistinguishable on Glass Identification at the reported precision. The study therefore supported SCG as the default trainer of the three methods. A tuned SGD remained adequate on well-separated classification data. LeapFrog required a careful cap on the step length.

The comparison was restricted to one hidden layer, to six small problems, and to a single architecture per problem. The listed restrictions bounded the claim to the protocol that was run.

REFERENCES

- [1] C. M. Bishop, *Neural Networks for Pattern Recognition*. Oxford: Oxford University Press, 1995.
- [2] A. P. Engelbrecht, *Computational Intelligence: An Introduction*, 2nd ed. Chichester: John Wiley & Sons, 2007.
- [3] J. H. Friedman, “Multivariate adaptive regression splines,” *The Annals of Statistics*, vol. 19, no. 1, pp. 1–67, 1991.
- [4] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [5] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, 2010, pp. 249–256.
- [6] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [7] W. H. Kruskal and W. A. Wallis, “Use of ranks in one-criterion variance analysis,” *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 583–621, 1952.
- [8] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, “Efficient backprop,” *Lecture Notes in Computer Science*, vol. 1524, pp. 9–50, 1998.
- [9] H. B. Mann and D. R. Whitney, “On a test of whether one of two random variables is stochastically larger than the other,” *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.
- [10] M. F. Møller, “A scaled conjugate gradient algorithm for fast supervised learning,” *Neural Networks*, vol. 6, no. 4, pp. 525–533, 1993.
- [11] B. T. Polyak, “Some methods of speeding up the convergence of iteration methods,” *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.
- [12] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning internal representations by error propagation,” in *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Volume 1: Foundations*. Cambridge: MIT Press, 1986, pp. 318–362.

- [13] S. S. Shapiro and M. B. Wilk, "An analysis of variance test for normality (complete samples)," *Biometrika*, vol. 52, no. 3–4, pp. 591–611, 1965.
- [14] J. A. Snyman, "A new and dynamic method for unconstrained minimization," *Applied Mathematical Modelling*, vol. 6, no. 6, pp. 449–462, 1982.
- [15] —, "An improved version of the original leap-frog dynamic method for unconstrained minimization: LFOP1(b)," *Applied Mathematical Modelling*, vol. 7, no. 3, pp. 216–218, 1983.

APPENDIX

LFOP	leap-frog optimiser
MSE	mean squared error
SCG	scaled conjugate gradient
SGD	stochastic gradient descent
UCI	University of California, Irvine
$\mathbf{a}$	LeapFrog acceleration, $\mathbf{a} = -\nabla E(\mathbf{w})$
α_k	SCG step length
$\mathbf{b}$	bias vector
β	SGD momentum coefficient
B	mini-batch
δ_k	SCG scaled curvature
$\delta_{\max}$	LeapFrog maximum step length
Δ_k	SCG comparison parameter
Δt	LeapFrog time step
E	training error
ε	logarithm offset in the cross-entropy
$\mathbf{h}$	hidden activation
η	SGD learning rate
λ	SCG scale parameter
$\bar{\lambda}$	SCG scale from the previous iteration
m	number of consecutive obtuse LeapFrog angles before Δt is halved
μ_k	SCG inner product $\mathbf{p}_k^\top \mathbf{r}_k$
n	number of training samples
n_h	number of hidden units
n_i	number of inputs
n_o	number of outputs
N	number of packed weights
$\mathbf{p}$	SCG search direction
$\mathbf{q}_k$	SCG Hessian-vector product
$\mathbf{r}$	steepest-descent vector, $\mathbf{r} = -\nabla E(\mathbf{w})$
s	mini-batch size
σ	SCG finite-difference interval
$\mathbf{u}$	SGD momentum buffer
$\mathbf{v}$	LeapFrog particle velocity
$\mathbf{w}$	packed weight vector
$\mathbf{W}$	weight matrix
$\mathbf{z}$	layer pre-activation